

returns the list without the first element. Notice how no attempt is made to restrict the list. Since we haven't actually tried to retrieve a value from this list, it will not be computed at all. If we were to call an element of the list (using `fib n !! <element number n>`), the list would then compute all elements upto and including the desired element, and return the `n`th element. Had the same logic been implemented in an eager or 'strict' language like as C or Java, it would've resulted in an infinite loop (or an exception, at best)

Hello World in Haskell:

Put the following line in file called `hello.hs` and run it with `ghc` (included in the DVD)

- `main = putStrLn "Hello, World!"`

Run it with the command `ghc -o hello hello.hs`

Features of Haskell

Defining functions in Haskell can be done in two ways, one where the single function takes multiple parameters (or arguments) and returns a value, and the other where a function acts on a single argument, which returns a function that will then take the second argument as its sole argument, and so on. The second approach is a more 'functional' way to approach programming and the result is more compact than the former. This approach is called currying, named after the American mathematician Haskell Curry.

Here is an example to illustrate the difference in the two approaches - the following function 'hyp' computes the value of the hypotenuse of a right angled triangle given the lengths of the two shorter sides.

Currying:

- `hyp :: Float -> Float -> Float`
- `hyp x y = sqrt (x*x + y*y)`

Non-Currying:

- `hyp :: (Float, Float) -> Float`
- `hyp (x,y) = sqrt (x*x + y*y)`



Relevant XKCD